

리눅스 3.2 - Tree와 데이터베이스 인덱스

남궁명수

[1] Tree

일반 이진 탐색 트리는 평균적으로 $O(\log N)$ 에 탐색할 수 있지만, 노드가 한쪽으로 치우치면 최악의 경우 $O(N)$ 이 된다.

균형 트리는 각 자료구조의 균형 규칙을 유지하여 트리 높이가 $O(\log N)$ 을 벗어나지 않도록 한다.

대표적으로 AVL Tree, Red-Black Tree, B-Tree 계열이 있으며, 탐색·삽입·삭제의 최악 시간 복잡도를 $O(\log N)$ 으로 보장한다.

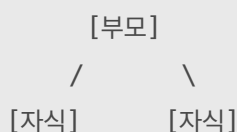
데이터베이스 인덱스에서는 단순한 시간 복잡도뿐 아니라 저장장치 페이지 I/O 횟수가 중요하다.

B-Tree 계열은 노드 하나에 많은 키와 자식 포인터를 저장하므로 트리 높이가 낮고, 한 번의 페이지 I/O로 많은 키를 확인할 수 있다.

또한 키가 정렬되어 있어 등가 검색뿐만 아니라 범위 검색과 정렬에도 효율적이다.

그래서 실제 데이터베이스 인덱스에는 B+Tree 또는 B*Tree 계열 구조가 널리 사용된다.

트리(Tree)는 노드들이 부모-자식 관계로 연결된 계층 자료구조다.



단, 모든 트리가 $O(\log N)$ 의 탐색 성능을 가지는 것은 아니다. 탐색 성능은 트리가 어떤 규칙으로 구성되어 있는지에 따라 달라진다.

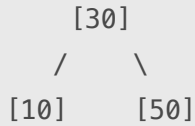
정렬 규칙이 없는 일반 트리에서는 원하는 값을 찾기 위해 모든 노드를 확인해야 할 수 있으므로 탐색 시간은 $O(N)$ 이다.

[2] 이진 탐색 트리(BST)

이진 탐색 트리(Binary Search Tree)는 다음 규칙을 갖는다.

왼쪽 서브트리의 값 < 부모의 값 < 오른쪽 서브트리의 값

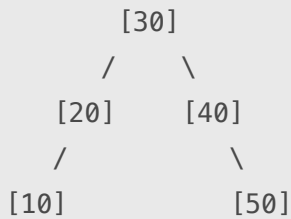
예를 들어, 다음과 같이 구성된다.



10을 찾는다면 30보다 작으므로 왼쪽으로 이동하고, 50을 찾는다면 오른쪽으로 이동한다.

탐색할 필요가 없는 절반의 영역을 제외할 수 있으므로 트리가 균형 있게 구성되어 있다면 탐색 시간은 평균적으로 $O(\log N)$ 이다.

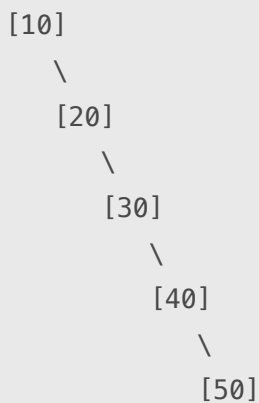
[균형 잡힌 경우]



트리의 높이가 낮게 유지되므로 탐색 시간이 $O(\log N)$ 이다.

[한쪽으로 치우친 경우]

10, 20, 30, 40, 50을 차례대로 삽입하면 다음과 같이 구성될 수 있다.



50을 찾으려면 모든 노드를 순서대로 확인해야 한다.

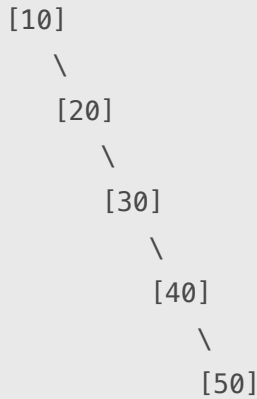
10 → 20 → 30 → 40 → 50

이 경우 트리의 높이가 N 에 가까워지므로 탐색 시간이 $O(N)$ 이 된다. 사실상 연결 리스트와 비슷한 구조가 된 것이다.

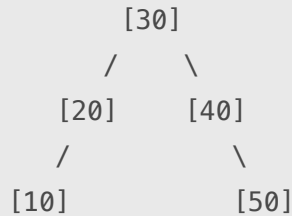
[3] 균형 트리(Balanced Tree)

균형 트리는 노드 삽입과 삭제 후 특정한 균형 규칙을 적용하여 트리의 높이가 지나치게 커지는 것을 방지하는 자료구조다.

한쪽으로 치우친 트리



균형 트리



균형 트리가 왼쪽과 오른쪽의 노드 개수를 항상 똑같이 만드는 것은 아니다.
각 자료구조가 정한 규칙을 통해 트리의 높이가 $O(\log N)$ 범위를 유지하도록 관리하는 것이 핵심이다.

대표적인 균형 트리는 다음과 같다.

AVL Tree
Red-Black Tree
B-Tree
B+Tree

[4] 데이터베이스가 B-Tree 계열을 사용하는 이유

균형 트리에는 Red-Black Tree도 있고, 평균적으로 $O(1)$ 탐색이 가능한 Hash Table도 있다.

그런데 데이터베이스 인덱스에서는 B-Tree 계열, 특히 B+Tree가 널리 사용된다.
그 이유는 단순히 B-Tree의 탐색 시간이 $O(\log N)$ 이기 때문만은 아니다.

데이터베이스에서는 다음 요소가 중요하다.

1. 저장장치 I/O 횟수
2. 트리의 높이
3. 범위 검색
4. 정렬된 데이터 순회
5. 삽입과 삭제 후의 균형 유지

[4-1] 데이터베이스와 저장장치

데이터베이스의 테이블과 인덱스는 기본적으로 SSD/HDD와 같은 영구 저장장치에 저장된다. 쿼리를 처리할 때 필요한 데이터베이스 페이지를 저장장치에서 RAM의 버퍼 풀로 가져온다.

SSD/HDD

- 테이블 데이터
- 인덱스 페이지

↓ 읽기

RAM의 DB Buffer Pool

↓

CPU가 탐색 및 비교

CPU가 B-Tree를 SSD/HDD 위에서 직접 연산하는 것은 아니다.

필요한 인덱스 페이지가 RAM에 없다면 저장장치에서 읽어 RAM으로 가져온 후, CPU가 RAM에 올라온 페이지의 키를 비교한다.

인덱스 루트 페이지 확인

↓

필요한 페이지가 RAM에 있는가?

└─ 있음 → 바로 탐색

└─ 없음 → SSD/HDD에서 읽어옴

↓

RAM에 저장

↓

CPU가 키 비교

이때 CPU의 키 비교보다 저장장치에 페이지를 읽는 I/O가 훨씬 비싸다.

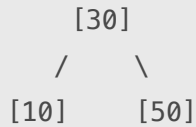
따라서 데이터베이스 인덱스에서는 단순한 키 비교 횟수보다 다음 질문이 더 중요하다.

원하는 데이터를 찾기 위해 저장장치 페이지를 몇 번 읽어야 하는가?

[4-2] Red-Black Tree를 사용하지 않는 이유

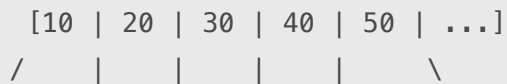
Red-Black Tree도 균형 트리이므로 탐색, 삽입, 삭제가 $O(\log N)$ 이다.

하지만 Red-Black Tree는 이진 트리이므로 노드 하나가 최대 두 개의 자식만 가진다.



한 단계마다 탐색 범위를 최대 두 개로만 나눌 수 있기 때문에 데이터가 많아지면 트리의 높이가 상대적으로 커진다.

반면 B-Tree는 노드 하나에 많은 키와 자식 포인터를 저장할 수 있다.



노드가 가질 수 있는 자식 수를 **분기 계수** 또는 **Fan-out**이라고 한다.

Red-Black Tree

→ 노드당 최대 자식 2개

B-Tree

→ 노드당 자식 수십~수백 개 가능

Fan-out이 커지면 한 단계마다 훨씬 넓은 범위를 제외할 수 있으므로 트리의 높이가 낮아진다.

Fan-out 증가

↓

트리 높이 감소

↓

탐색하면서 읽는 페이지 수 감소

↓

저장장치 I/O 감소

예를 들어, 약 1억개의 데이터를 관리한다고 단순하게 가정해보자.

이진 트리

$\log_2(100,000,000)$

≈ 27단계

자식이 500개인 B-Tree

$\log_{500}(100,000,000)$

≈ 3단계

실제 높이는 키와 페이지 크기, 노드, 노드 사용률 등에 따라 달라지지만 B-Tree가 이진 트리보다 훨씬 낮은 높이를 유지한다는 점이 핵심이다.

따라서 B-Tree는 적은 페이지 I/O만으로도 원하는 키에 도달할 수 있다.

[4-3] Hash Table을 사용하지 않는 이유

Hash Table은 특정 키와 정확히 일치하는 데이터를 찾는 등가 검색에 매우 효율적이다.

```
SELECT *  
FROM users  
WHERE id = 100;
```

```
hash(100)  
  ↓  
Bucket 위치 계산  
  ↓  
해당 Bucket 탐색
```

평균 탐색 시간은 $O(1)$ 이다.

하지만 해시 함수는 일반적으로 원래 키의 정렬 순서를 유지하지 않는다.

```
원래 키:  
10, 20, 30, 40, 50  
  
해시 결과:  
7, 2, 9, 1, 4
```

따라서 다음과 같은 범위 검색과 정렬 작업에는 적합하지 않다.

```
WHERE age BETWEEN 20 AND 30;  
  
WHERE age > 20;  
  
ORDER BY age;  
  
SELECT MIN(age);  
  
SELECT MAX(age);
```

B-Tree 계열은 키를 정렬된 상태로 저장하기 때문에 특정 범위의 시작점을 찾은 후
순서대로 데이터를 읽을 수 있다.

정렬된 키

[10 | 20 | 30 | 40 | 50]

20 이상 40 이하

↓

20 → 30 → 40